SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA0307	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Charandeep.N.C	**Reg. No.:**	192472196
Section / Batch:	CSE-AI	**Date:**	8-8-2026

Code of Conduct

*I __**Charandeep.N.C (192472196)**__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Charandeep**

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2

Probabilistic Context-Free Gra ımars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Annexure A – Experiments

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype def parse_cfg(sentence):
pass

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Aim

To design a Python program that validates whether a sentence follows the given Context-Free Grammar (CFG) rules and generates its parse tree representation.

Algorithm

1. Import NLTK.

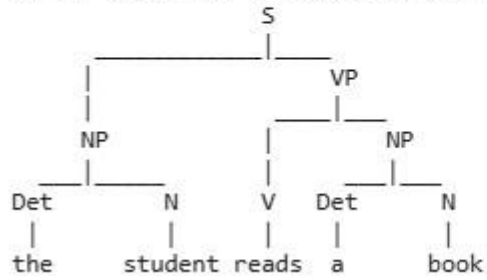
2. Define CFG grammar.
3. Accept sentence input.
4. Tokenize the sentence.
5. Parse using RecursiveDescentParser.
6. If parsing succeeds:
    - Print "Valid Sentence"
    - Display Parse Tree
7. Otherwise print "Invalid Sentence".

Python Code

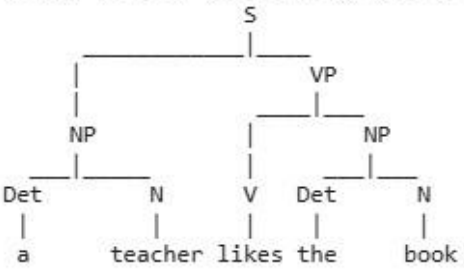
```
import nltk from nltk
import CFG
from nltk.parse import RecursiveDescentParser

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'student' | 'teacher' | 'book'
V -> 'reads' | 'likes'
""")

parser = RecursiveDescentParser(grammar)

sentence = input("Enter sentence: ").lower().split()

parsed = False

for tree in parser.parse(sentence):
parsed = True     print("\nValid
Sentence")     print("\nParse
Tree:")     print(tree)
tree.pretty_print()

if not parsed:
print("\nInvalid Sentence")
```

Outputs:

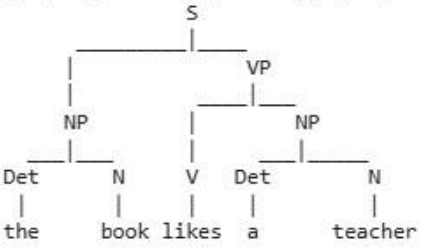
```
Enter sentence: the student reads a book

Valid Sentence

Parse Tree:
(S (NP (Det the) (N student)) (VP (V reads) (NP (Det a) (N book))))
```




```
Enter sentence: a teacher likes the book

Valid Sentence

Parse Tree:
(S (NP (Det a) (N teacher)) (VP (V likes) (NP (Det the) (N book))))
```




```
Enter sentence: the book likes a teacher

Valid Sentence

Parse Tree:
(S (NP (Det the) (N book)) (VP (V likes) (NP (Det a) (N teacher))))
```




```
Enter sentence: reads the student book

Invalid Sentence
```

Result Analysis

The parser successfully validates sentences according to the CFG rules. Valid sentences produce a complete parse tree, while sentences violating grammar rules are rejected.

Result

Thus, the Python program successfully validates CFG sentences and constructs parse trees.

Experiment 2: Agreement and Feature Structure Checking Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

he → singular, third person
she → singular, third person it
→ singular, third person they
→ plural

runs → singular verb
writes → singular verb

run → plural verb
write → plural verb

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
pass
```

Test Cases Subject Verb Expected

he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Aim

To verify subject–verb agreement using feature structures based on Number and Person.

Algorithm

1. Store feature structures for subjects.
2. Store feature structures for verbs.
3. Read subject and verb.
4. Compare Number feature.
5. Print True if matched.
6. Else print False.

Python Code

```
subjects = {

   "he": ("singular", "third"),
   "she": ("singular", "third"),

   "it": ("singular", "third"),

   "they": ("plural", "third")

} verbs =

{

   "runs": "singular",

   "writes": "singular",

   "run": "plural",

   "write": "plural" } def

check_agreement(subject, verb):     if subject

not in subjects or verb not in verbs:

       return False     subject_number =

subjects[subject][0]     verb_number =

verbs[verb]     return subject_number ==

verb_number subject = input("Enter

subject: ").lower() verb = input("Enter

verb: ").lower()

print(check_agreement(subject, verb))
```

Outputs:

```
Enter subject: he
Enter verb: runs
True
```

```
Enter subject: he
Enter verb: run
False
```

```
Enter subject: they
Enter verb: run
True
```

```
Enter subject: they
Enter verb: writes
False
```

```
Enter subject: she
Enter verb: writes
True
```

Result Analysis

The program compares the grammatical number of the subject and verb using feature structures. It correctly identifies agreement and disagreement.

Result

Thus, the Python program successfully verifies subject–verb agreement using feature structures.

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

S → NP VP [1.0]

NP → John [0.6]
NP → Mary [0.4]

VP → runs [0.5]
VP → walks [0.5]

Probability Formula

P(Sentence) = P(S→NP VP) × P(NP) × P(VP) **Task**

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
pass
```

Test Cases Input Sentence Expected Probability

John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Aim

To compute sentence probability using Probabilistic Context-Free Grammar rules.

Algorithm

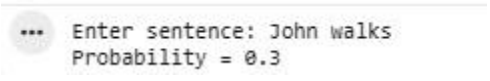
1. Define PCFG probabilities.
2. Accept a two-word sentence.
3. Split into NP and VP.
4. Multiply grammar probabilities.
5. Print probability.
6. If sentence cannot be generated, return 0.

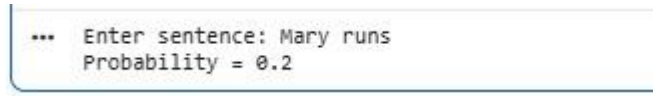
Python Code

```
np_prob = {

    "John": 0.6,

    "Mary": 0.4
```

```
} vp_prob =
{
   "runs": 0.5,
   "walks": 0.5
} def
sentence_probability(sentence):
   words = sentence.split()
if len(words) != 2:
      return 0.0
np = words[0]
vp = words[1]
if np not in
np_prob:
      return 0.0     if vp
not in vp_prob:
      return 0.0     probability = 1.0 * np_prob[np] *
vp_prob[vp]     return probability sentence =
input("Enter sentence: ") print("Probability =",
sentence_probability(sentence))
```

Outputs:





```
... Enter sentence: Peter runs
    Probability = 0.0
```

Result Analysis

The program computes sentence probabilities by multiplying the production rule probabilities. Invalid words receive a probability of 0, indicating that the sentence cannot be generated by the grammar.

Result

Thus, the Python program successfully computes sentence probabilities using PCFG production rules.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Document ation	Sub. Total	Total %
1							

2							
3							

Faculty In-charge	**Course Coordinator**	**Program Director**
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____